



1 ML Essentials

1.1 What is ML?

• setting:

- Features (x): quantities that are easy to measure ("inputs", "covariates", "predictors")
- Response (y): quantities we wish to predict ("labels", "outputs")
- We aim to find a function f such that
$$f(x) = \hat{y} \approx y^*$$
 - $\hat{y}$ = prediction
 - y^* = true value
- Key idea: we use data to learn a mapping from features to responses.

• Traditional approach: handcraft $f(x)$

• ML approach:

1) Step 1: Choose a universal function family $F(x; \theta)$ that can represent many possible functions in Θ

2) Step 2: Use training data to find the best parameters $\hat{\theta}$ such that

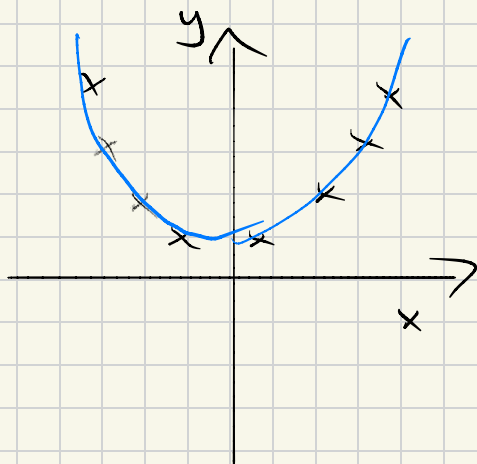
$$f(x; \hat{\theta}) \approx y^*$$

Example

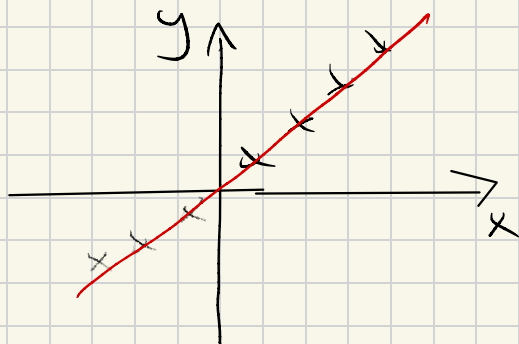
$$F(x; \theta = (a, b, c)) = ax^2 + bx + c$$

$$x, y \in \mathbb{R}$$

Data 1



$$\begin{aligned} a &= 1 \\ b &= 0 \\ c &= \text{small} \end{aligned}$$



$$\begin{aligned} a &= 0 \\ b &= 1 \\ c &\approx 0 \end{aligned}$$

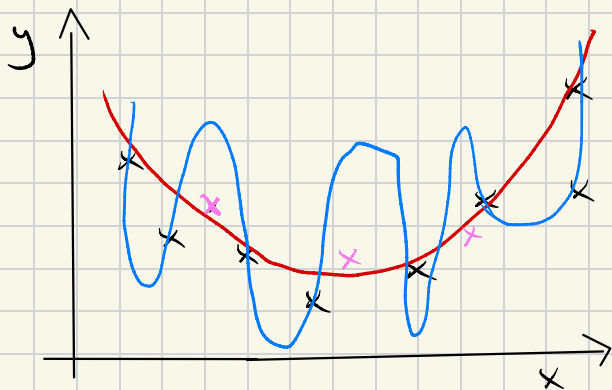
1.2 Basic Machine Learning Workflow

1) Data Collection

- Training Data : Used to learn $\hat{\theta}$
- Test Data : Used to evaluate generalization

2) Training data: Find $\hat{\theta}$ so that $F(x; \hat{\theta}) \approx y^*$ on the training data

3) Testing : Evaluate performance on new data to avoid overfitting. Fix $\hat{\theta}$, we evaluate $\hat{y} = F(x, \hat{\theta}) \approx y^*$ in our test set. If not (bad generalization), throw away and retry. (validation data set required)



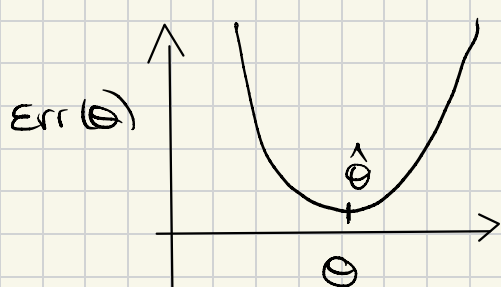
F_1 : no perfect interpolation, but good at generalizing

F_2 : perfect interpolation, bad generalization

- 4) Deployment: Use the trained model $F(x; \hat{\theta})$ to predict new data in real-world settings.

1.3 Loss Landscape

- Say we have a model F , parametr. by θ
- Goal: Find θ to minimize some notion of error/loss



1.4 The argmax Operator

- Suppose we have a function $u = f(v)$
- then the max operator returns max. value of f : $u_{\max} = \max_v f(v)$
- argmax returns the argument to attain max:
 $v_{\max} = \operatorname{argmax}_v f(v)$



1.5 Notation

- Each observation is called instance
- The training set (data) contains N instances:
 (x_i, y_i) for $i = 1 \dots N$ (or i', i'')
- Features: x_i is a D -dimensional vector, $x_i \in \mathbb{R}^D$
 indexed by x_{ij} , $j \in 1 \dots D$ (j', j'')
 "x_{ij} is feature j in instance i "
- Response $y_i \in \mathbb{R}^{D'}$ (typically $D' = 1$)
- Variables can be:
 - continuous (real-valued)
 - discrete
 - * ordinal data (likest scales: small < med. < large)
 - * categorical data (unordered: cat, dog, human, ...)

• Two fundamental ML problem types:

- regression: y is continuous
- classification: y is discrete
- if y is discrete: C is the number of possible class labels, indexed by $k \in 1 \dots C$

Types of training sets: supervised, unsupervised, or weakly supervised learning

- supervised learning: each instance has both features and a response (x_i, y_i)
- unsupervised learning: no responses y are provided

1.6 OVERVIEW ML

MACHINE LEARNING

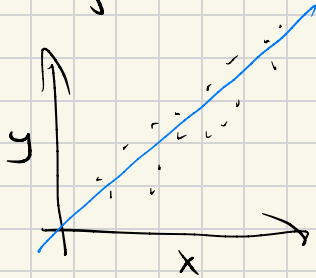
SUPERVISED LEARNING

"Model trained with labeled data"

Classification



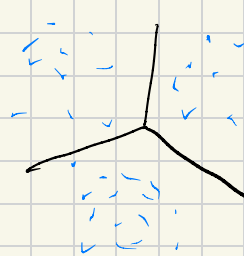
Regression



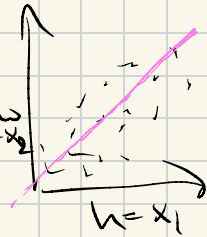
UNSUPERVISED LEARNING

"Model training w/o labeled data"

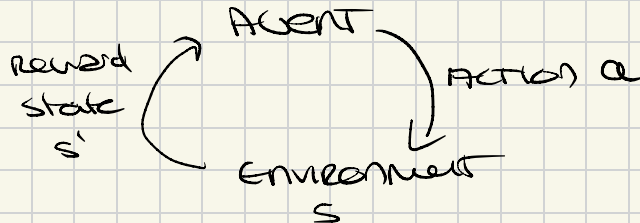
CLUSTERING



DIMENSIONALITY REDUCTION



REINFORCED LEARNING



1.7 TWO TYPES OF LOSS FUNCTIONS:

DETERMINISTIC VS PROBABILISTIC MODEL

- DETERMINISTIC MODEL:

$$\hat{y} = f(x)$$

- PROBABILISTIC MODEL

$$p(y | x)$$

- SOURCES OF UNCERTAINTY

- 1) Ambiguity: features may not fully determine y
- 2) Aleatoric uncertainty: noise inherent in our measurement; irreducible randomness
- 3) Epistemic Uncertainty: uncertainty due to lack of perfect knowledge

Example: coin toss

- response $y = 1$ (heads), $y = 0$ (tails)

$$p(y=1 | x)$$

x = properties of the coin (fair)

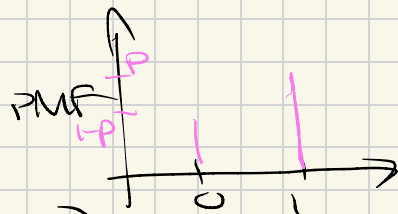
- binary variables can be modeled as

Bernoulli random variables

$$y \sim \text{Ber}(p(x))$$

- $k=1$ ("success"), $k=0$ ("failure")

$$p^k (1-p)^{1-k}$$



$$p(x) = \frac{1}{1 + e^{\theta \cdot x}} \in (0, 1)$$

1.8 From Probabilistic Answers to Hard Decision

Common Methods:

- 1) Choose the most probable answer (mode),
Maximum a-posteriori (MAP) estimate:

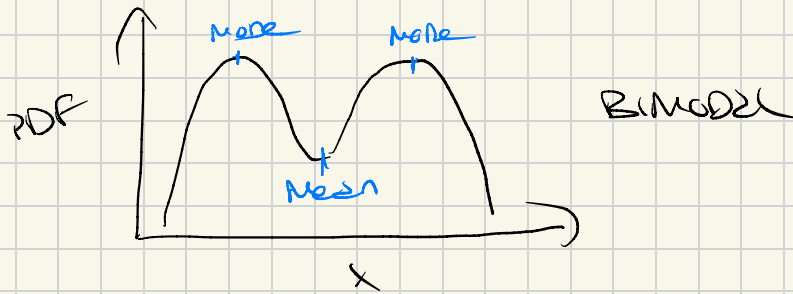
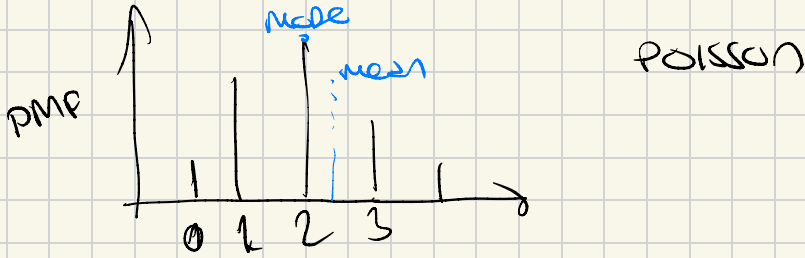
$$\hat{y} = \underset{y}{\operatorname{argmax}} p(y | x; \hat{\theta})$$

- 2) Choose the expected value

$$\mathbb{E}_{y \sim p_{\hat{\theta}}(y|x)}[y] = \begin{cases} \sum_y y \cdot p_{\hat{\theta}}(y|x) & \text{if } y \text{ discrete} \\ \int_y y \cdot p_{\hat{\theta}}(y|x) dy & \text{if } y \text{ is continuous} \end{cases}$$

- 3) Choose the median

$$\hat{y} = y \text{ such that } p(y < \hat{y}(x)) = p(y > \hat{y}(x))$$



SUPERVISED LEARNING

2 CLASSIFICATION

- y is discrete

$$y \in \{1, 2, 3, \dots, C\}$$

- Binary classification

$$y \in \{0, 1\} \quad y \in \{-1, 1\}$$

- Objective: learn a function $\hat{f}$ such that

$$\hat{y} = \hat{f}(x) \approx y^*$$

2.1 Measuring classification performance

- use confusion Matrix

		Prediction	
		(1) Positive	(-1) Negative
True Data	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

Error Rates

1) TOTAL ERROR RATE

$$\text{Error} = \frac{\#FP + \#FN}{N}$$

$$N = \#FP + \#FN + \#TP + \#TN$$

2) FALSE POSITIVE RATE

$$\text{FPR} = \frac{\#FP}{\#FP + \#TN}$$

3) FALSE NEGATIVE RATE

$$\text{FNR} = \frac{\#FN}{\#TP + \#FN}$$

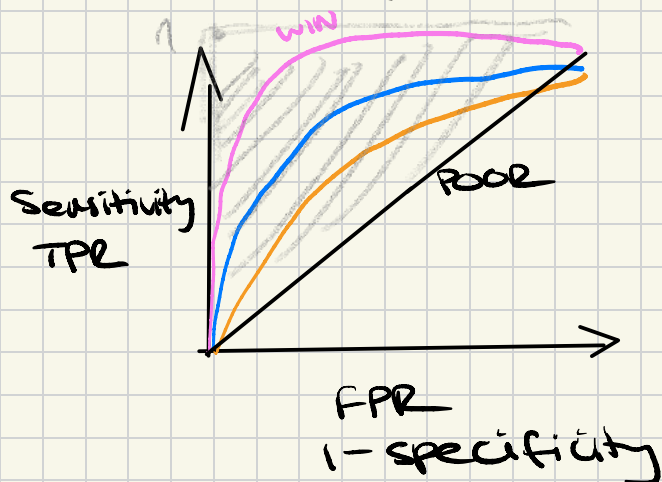
4) Sensitivity (Recall / TPR)

$$\text{sensitivity} = \frac{\#TP}{\#TP + \#FN}$$

5) Specificity

$$\text{Spec.} = \frac{\#TN}{\#TN + \#FP}$$

6) AUROC : the area under the Receiver Operating Characteristic curve



2.2 The threshold classifier

- 1-D threshold classifier

$x \in \mathbb{R}$, $y \in \{-1, 1\}$, $x_0 = \text{threshold}$

- Define
$$\hat{y} = \text{sign}(x - x_0) = \begin{cases} +1 & \text{if } x > x_0 \\ -1 & \text{if } x < x_0 \end{cases}$$

Example

- $x = \text{weight}$
- $x_0 = 100 \text{ kg}$
- Decide : if $x > 100 \Rightarrow \text{"overweight"}$

Generalizing to Multiple Features

- Three solutions

1) Linear Scoring Function

- reduce $x \in \mathbb{R}^D$ to a score:

$$z = g(x) = \sum_{j=1}^D \beta_j x_j \in \mathbb{R}$$

- if g is nonlinear, then classifier is nonlinear.

- Apply threshold principle to z :

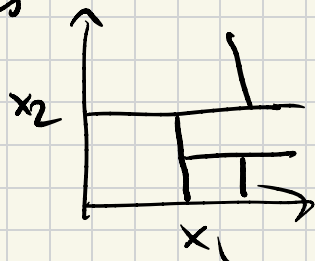
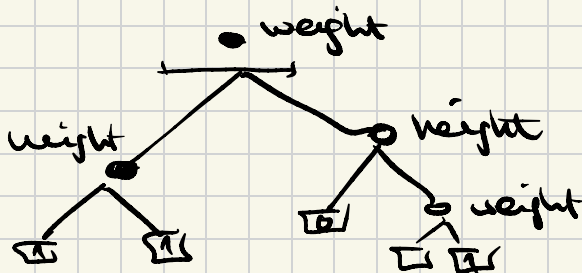
$$\hat{y} = \text{sign}(z - x_0)$$

- Example

$$\text{BMI} = \frac{\text{weight}}{\text{height}^2} = \frac{x_1}{x_2^2}$$

2) Decision tree

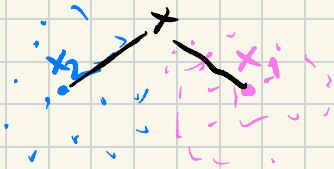
- Reduce the multi-dim problem to a sequence of 1-D problems



3) Nearest Neighbor Classification

- Define prototypes / representatives for each class (e.g. mean / median)
- Assign the test point to label of the closest representative

$$\hat{i} = \underset{i}{\operatorname{argmin}} d(x, x_i) \quad \hat{y} = y_i$$



2.3 Nearest Neighbor Classification

- Nearest Neighbor Rule:

- Given a training set $\{(x_i, y_i)\}_{i=1}^N$,
predict

$$\hat{i} = \underset{i}{\operatorname{argmin}} d(x, x_i)$$
$$\hat{y} = y_i$$

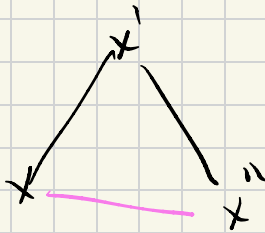
- Properties of d :

1) $d(x, x) = 0$

2) $d(x, x') > 0$ if $x \neq x'$

3) Triangle inequality:

$$d(x, x') + d(x', x'') \geq d(x, x'')$$



• Common Distance Functions

• Euclidean Distance

$$d(x, x') = \sqrt{\sum_{j=1}^D (x_j - x'_j)^2}$$

• Weighted Euclidean Distance

$$d(x, x') = \sqrt{\sum_{j=1}^D w_j (x_j - x'_j)^2}$$

• $w_j = \frac{1}{\sigma_j^2} \rightarrow$ variance of feature j

2.4 Perceptron

- A linear classific. fun'n :

$$\hat{y} = \text{sign} \left(\sum_{j=1}^D w_j x_{ij} + b \right)$$

learnable parameters

Trick to get rid of b :

- 1) we can centralize our features by subtracting mean

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\tilde{x}_i = x_i - \bar{x}$$

- 2) Absorb b into feature/param. vector

$$\tilde{x}_{i,0} = 1$$

$$\sum_{j=0}^D w_j x_{ij} = \sum_{j=1}^D w_j x_{ij} + b$$

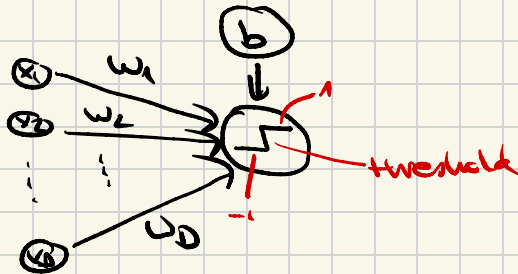
vector notation:

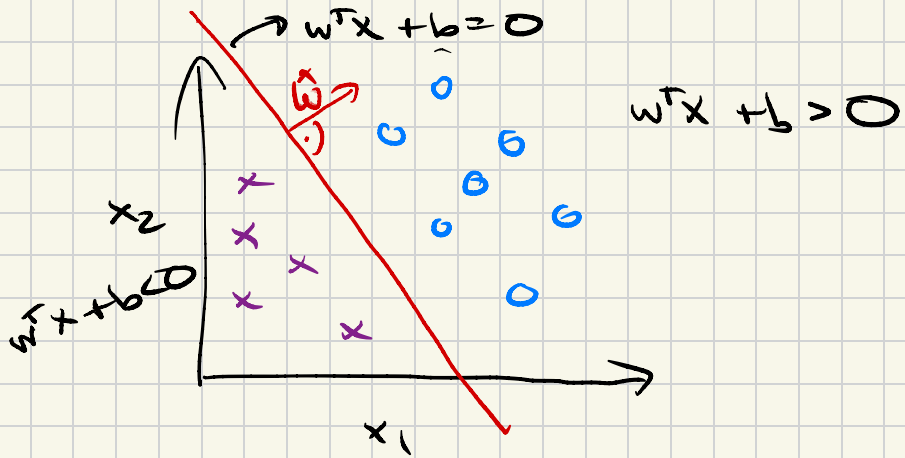
$$w^T x = [b \ w_1 \ \dots \ w_D] \begin{bmatrix} 1 \\ x_{i1} \\ \vdots \\ x_{iD} \end{bmatrix}$$

2.4.1 Perceptron Algorithm (1957, Rosenblatt)

- setting: • training data $\{(x_i, y_i)\}_{i=1}^N$; $x_i \in \mathbb{R}^D$
- $y_i \in \{-1, 1\}$

$$\hat{y} = f(x) = \text{sign}(w^T x + b)$$





$y_i [w^T x] > 0$ defines points that are correctly classified

loss function : 0/1 loss

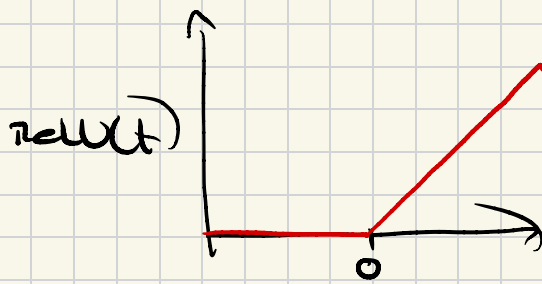
loss function $\mathcal{L}(\hat{y}_i, y_i^*) = \begin{cases} 0 & \text{if } \hat{y}_i = y_i^* \\ 1 & \text{if } \hat{y}_i \neq y_i^* \end{cases}$

2.4.2 Perceptron loss

$$\mathcal{L}(\hat{y}_i, y_i^*) = \begin{cases} 0 & \hat{y}_i = y_i^* \\ \text{ReLU}(-y_i w^T x_i) & \text{if } \hat{y}_i \neq y_i^* \end{cases}$$

$$\text{ReLU} = \max(0, t) = \begin{cases} 0 & \text{if } t < 0 \\ t & \text{if } t \geq 0 \end{cases}$$

Rectified linear unit

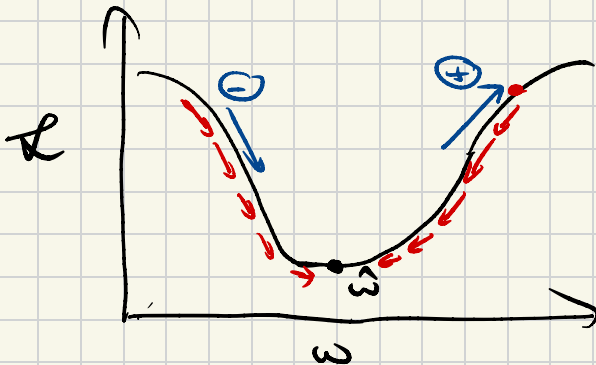


Active: $-y_i w^T x_i > 0$ for misclassifications

2.4.3 Gradient Descent for the Perceptron

$$w^{(t)} = w^{(t-1)} - \tau \frac{\partial \mathcal{R}}{\partial w} \rightarrow \text{all param.}$$

- $w^{(t)}$ = updated weight vector
- $w^{(t-1)}$ = previous weight vector
- τ = learning rate ($\tau < 1$), $\tau > 0$
- $\frac{\partial \mathcal{R}}{\partial w}$ = gradient of the loss function



- The gradient of the perceptron loss

$$\frac{\partial \mathcal{L}}{\partial \omega} = \frac{\partial \text{ReLU}(-y_i \omega^T x_i)}{\partial \omega} = \begin{cases} 0 & \text{if } -y_i \omega^T x_i < 0 \\ -y_i x_i^T & \text{if } -y_i \omega^T x_i > 0 \end{cases}$$

correct

- Perceptron update rule:

$$\omega^{(t)} = \omega^{(t-1)} + \tau y_i x_i$$

Algorithm

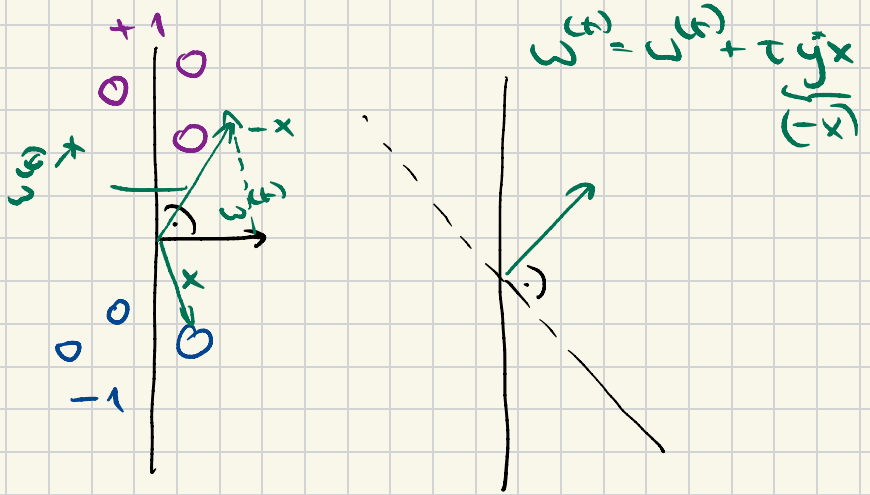
- 1) Initialize: $\omega^{(0)}$ (sample from standard normal or a uniform distr. $[-1, 1]$)
- 2) Iterative update rule

for $t = 1 \dots T$ (or until convergence)

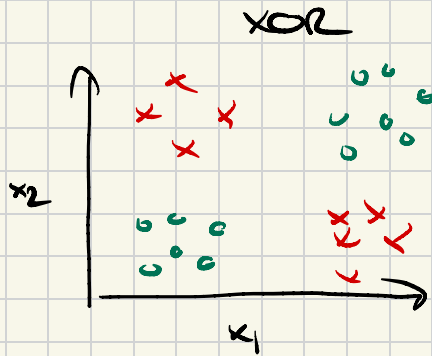
$$\omega^{(t)} = \omega^{(t-1)} + \tau \frac{1}{N} \sum_{i: \hat{y}_i^{(t-1)} \neq y_i^*} y_i x_i$$

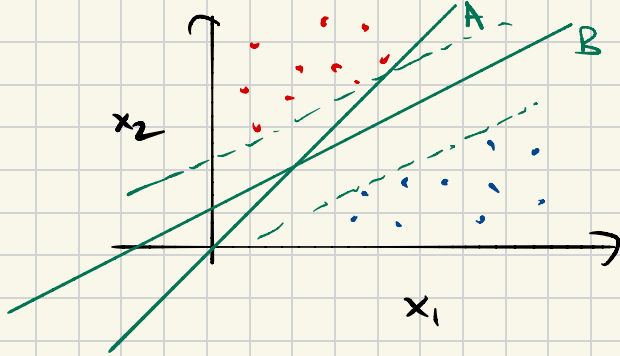
↑
learning rate

$$\hat{y}_i^{(t-1)} = \text{sign}(\omega^{(t-1)T} x_i)$$



- **Convergence Theorem:** If the training data is linearly separable, the perceptron algo. will converge in a finite number of steps.
- - does not converge if data is not separable
- - finds any hyperplane that separates the data





- margin: "measure of confidence"; makes classifier less sensitive to perturbations

2.5 Support Vector Machines (SVMs)

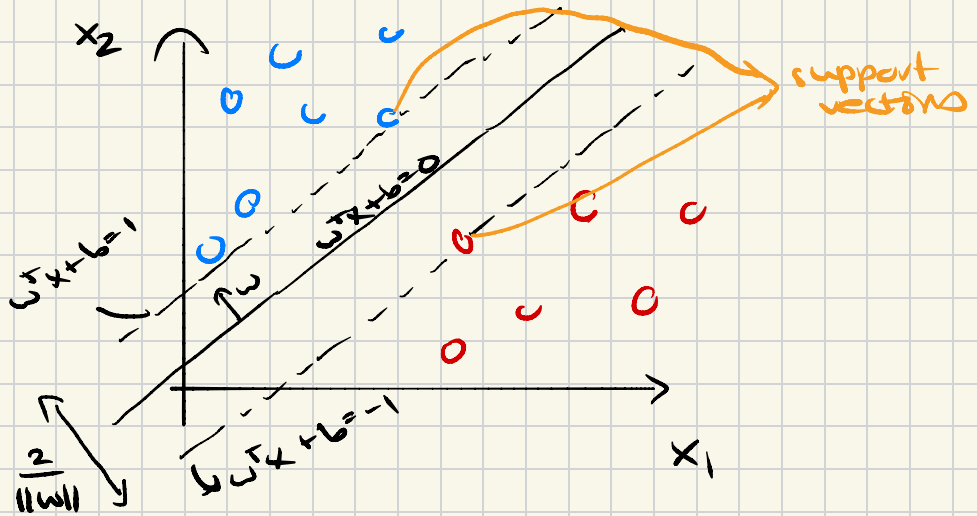
2.5.1. Hard-Margin SVM

- Assumption: Data is linearly separable
- ~~idea~~: seek a hyperplane $f(x) = w^T x + b$:

$$y_i (w^T x + b) \geq 1 \quad \forall i$$
- Optimization Problem:

$$\min \frac{1}{2} \|w\|^2 \quad \text{subject to } y_i (w^T x_i + b) \geq 1 \quad \forall i$$

minimizing this maximizes margin



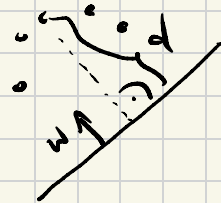
- Defined 2 add. hyperplanes:

$$w^T x + b - 1 = 0$$

$$w^T x + b + 1 = 0$$

- The perpendicular distance d of any point x from the hyperplane $w^T x + b = 0$:

$$d = \frac{|w^T x + b|}{\|w\|}$$



- For the support vectors

$$|w^T x_i + b| = 1$$

- The minimum distance from the hyperplane